



INSTITUTO FEDERAL DA PARAÍBA
UNIDADE ACADÊMICA DE INFORMAÇÃO E COMUNICAÇÃO COORDENAÇÃO
DO CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET

RELATÓRIO DE ESTÁGIO

SISTEMA INTEGRADO DE BACKEND E AUTOMAÇÃO PARA
CERTIFICAÇÃO DE SOFT SKILLS NO PROGRAMA
CAPACITAÇÃO 4.0

Gabriel Macaúbas Melo

João Pessoa – PB

2025

Instituto Federal de Educação, Ciência e Tecnologia da Paraíba
Unidade Acadêmica de Informação e Comunicação
Coordenação do CST em Sistemas para Internet

**SISTEMA INTEGRADO DE BACKEND E AUTOMAÇÃO PARA
CERTIFICAÇÃO DE SOFT SKILLS NO PROGRAMA
CAPACITAÇÃO 4.0**

Gabriel Macaúbas Melo

Relatório de Estágio Supervisionado
apresentado à Coordenação do Curso Superior
de Tecnologia em Sistemas para Internet do
Instituto Federal de Educação, Ciência e
Tecnologia da Paraíba, como requisito parcial
para obtenção do grau de Tecnólogo em
Sistemas para Internet.

Orientador: Dra. Heremita Brasileiro Lira
Coordenador do Curso: Ms. Candido José Ramos Do Egypto
Empresa: IFPB
Período: 01/03/2024 a 31/12/2025

APROVAÇÃO

Prof. Dra. Heremita Brasileiro Lira

Professora Supervisora/Orientadora

Prof. Ms. Candido José Ramos do Egypto

Coordenador do CST de Sistemas para Internet

Gabriel Macaúbas Melo

Aluno

AGRADECIMENTOS

Gostaria de agradecer primeiramente a minha família, especialmente à minha mãe e meu irmão que me apoiaram desde a adolescência quando decidi enfrentar a carreira tecnológica. Todo o amor, apoio e ensinamento de vocês foi fundamental para que eu chegasse até aqui.

Aos amigos que fiz nessa jornada, agradeço eternamente pelos momentos de alegria e parceria que dividimos ao longo dos anos, tudo seria mais difícil sem vocês.

Aos professores, em especial à professora Heremita, que me guiou, orientou e abriu portas na minha carreira profissional e acadêmica. Obrigado pela confiança dada ao longo deste projeto de pesquisa.

Por fim, ao São Paulo Futebol Clube, que sempre esteve ao meu lado durante todos os momentos, bons e ruins. Minha referência de trabalho, esforço, dedicação e conquistas.

“É impossível alcançar a perfeição, mas é possível aproximar-se dela.”

(SANTANA, Telê, 1992).

RESUMO

Este relatório apresenta as atividades desenvolvidas no âmbito do projeto de pesquisa “Automação de Recomendações e Certificações de Soft Skills em Programas de Capacitação Profissional”, vinculado ao Polo de Inovação do IFPB e apoiado pela EMBRAPIL. O trabalho representa a continuidade das ações iniciadas em 2024, focadas na otimização de processos de gerenciamento do projeto Capacitação 4.0. A atividade central consistiu na evolução de uma *API Restful*, desenvolvida com o *framework Django 4*, que serviu como núcleo de persistência e processamento para dados de competências socioemocionais. A infraestrutura tecnológica incluiu o uso de banco de dados *PostgreSQL*, containerização com *Docker* e deploy na plataforma *Vercel* para garantir a escalabilidade da solução. Por meio da integração sistêmica com a ferramenta *n8n*, foram orquestrados fluxos automatizados para a geração de gráficos radar, emissão de certificados e fornecimento de recomendações pedagógicas personalizadas baseadas no desempenho discente. A implementação resultou na centralização da gestão de dados e na redução do esforço manual, assegurando segurança via protocolos *JWT* e suporte robusto à tomada de decisão pedagógica.

Palavras-chave: API Django, Automação n8n, Soft Skills, Backend, Certificação Digital.

LISTA DE FIGURAS

Figura 1: Diagrama Entidade-Relacionamento da estrutura do banco de dados	22
Figura 2: Estrutura de tabelas nativas do Django para suporte e governança do sistema	23
Figura 3: Composição técnica de Model e ViewSet da API Django	26
Figura 4: Interface de documentação técnica e endpoints da API Django	28
Figura 5: Fluxo de automação no n8n integrado aos endpoints da API Django	30
Figura 6: Configurações de Docker	32
Figura 7: Parâmetros de deploy na plataforma Vercel	33

LISTA DE SIGLAS E ABREVIATURAS

ACID	Atomicidade, Consistência, Isolamento e Durabilidade
API	Interface de Programação de Aplicações
EMBRAPII	Empresa Brasileira de Pesquisa e Inovação Industrial
HTTP	Protocolo de Transferência de Hipertexto
IFPB	Instituto Federal da Paraíba
JSON	Notação de Objetos JavaScript
JWT	Token Web em formato JSON
N8N	Ferramenta de Automação
ORM	Mapeamento Objeto-Relacional
REST / Restful	Transferência Representacional de Estado
SGBD	Sistema Gerenciador de Banco de Dados
SQL	Linguagem de Consulta Estruturada

SUMÁRIO

LISTA DE FIGURAS	7
LISTA DE SIGLAS E ABREVIATURAS	8
1. Introdução	10
1.1 Objetivo Geral	10
1.2 O Projeto	11
1.3 Descrição Geral das Atividades	11
1.4 Organização do Relatório	11
2. Embasamento Teórico	12
2.1 O Framework Django e o Ecossistema Python	12
2.2 Arquitetura e Desenvolvimento de APIs Restful	12
2.3 Docker	13
2.4 PostgreSQL	14
2.5 Integração de Sistemas e Ferramenta n8n	15
3. Atividades Realizadas	16
3.1 Gerenciamento do Projeto	16
3.2 Planejamento e Levantamento de Requisitos	16
3.3 Modelagem e Arquitetura do Banco de Dados	18
3.4 Desenvolvimento do Backend e Lógica de Negócio	20
3.5 Segurança, Autenticação e Documentação da API	23
3.6 Integração com n8n e Automação de Fluxos	25
3.7 Infraestrutura, Containerização e Deploy	27
4. Considerações Finais	31
Referências	34

1. Introdução

A otimização de processos e o aumento da eficiência operacional constituem desafios centrais para organizações contemporâneas, muitas vezes dificultados pela elevada carga de atividades repetitivas que resulta em fluxos de trabalho extensos e atrasos na entrega de produtos. No âmbito dos processos de avaliação de soft-skills no projeto de Capacitação 4.0, a automação de atividades repetitivas torna-se essencial para simplificar tarefas operacionais e promover a padronização e confiabilidade dos processos de avaliação de soft skill. Este relatório descreve as atividades desenvolvidas em um projeto de pesquisa voltado ao monitoramento de competências socioemocionais (*Soft Skill*) de discentes vinculados ao Programa de Capacitação 4.0 do Polo de Inovação do IFPB, em parceria com a EMBRAPPII. A principal atividade concentrou-se na evolução de uma *API Restful* desenvolvida com o *framework Django 4*, que atuou como núcleo de processamento e persistência de dados para as automações executadas via *n8n* durante os anos de 2024 e 2025 do Capacitação 4.0.

1.1 Objetivo Geral

O objetivo central deste trabalho consiste no desenvolvimento e aprimoramento da infraestrutura de backend para o monitoramento de soft skills no contexto de programas de capacitação profissional, visando centralizar a gestão de dados e automatizar processos de apoio à decisão pedagógica.

Objetivos Específicos

- Identificar as necessidades técnicas e as regras de negócio do programa Capacitação 4.0 por meio de reuniões de planejamento e análise do manual da EMBRAPPII, afim de levantar os requisitos necessários.
- Estruturar a arquitetura lógica do banco de dados *PostgreSQL* para assegurar a integridade e a rastreabilidade histórica das avaliações dos discentes.
- Construir a lógica de negócio e os endpoints da *API Django* para automatizar o processamento e a conversão de notas numéricas em níveis qualitativos.
- Implementar protocolos de autenticação via tokens *JWT* e documentar a interface da *API* para garantir o acesso controlado e a interoperabilidade entre sistemas.
- Orquestrar e testar fluxos de trabalho na ferramenta *n8n* para a geração automática de certificados, gráficos radar e recomendações pedagógicas personalizadas.

- Configurar o ambiente de containerização com *Docker* e realizar o deploy na plataforma *Vercel* para garantir a estabilidade e escalabilidade da solução em produção.

1.2 O Projeto

Denominado Automação de Recomendações e Certificações de Soft Skills em Programas de Capacitação Profissional, o projeto está vinculado ao Polo de Inovação do IFPB e conta com o apoio da EMBRAPPII. A iniciativa representa a continuidade das atividades iniciadas em 2024, focadas na automação de processos repetitivos em gerenciamento do projeto Capacitação 4.0, mais especificamente no processo de avaliação de *soft skills*. O projeto utiliza a metodologia de aprendizagem baseada em problemas para fortalecer a formação técnica e profissional dos estudantes, garantindo que os mentores disponham de relatórios precisos sobre o progresso socioemocional dos discentes através da automação de processos que facilitam a tomada de decisão pedagógica.

1.3 Descrição Geral das Atividades

As atividades desempenhadas ao longo do estágio envolveram o ciclo completo de desenvolvimento de software e a integração de sistemas complexos. No que tange ao desenvolvimento *backend*, foram estruturados modelos de dados e *serializers* para o gerenciamento de entidades como discentes e avaliações. A gestão do banco de dados abrangeu a manutenção de registros históricos para viabilizar a comparação de desempenho entre diferentes ciclos avaliativos ao longo de dois anos. Complementarmente, foram construídas rotas para o cadastro de notas e procedimentos de

1.4 Organização do Relatório

O presente documento está organizado em quatro capítulos que descrevem a trajetória técnica percorrida. O primeiro capítulo introduz o contexto e define os objetivos do projeto de estágio. O segundo capítulo apresenta o embasamento teórico, discutindo os conceitos de automação e as tecnologias que fundamentam as soluções desenvolvidas. O terceiro capítulo descreve detalhadamente as atividades realizadas, com ênfase no desenvolvimento da *API Django* e sua integração técnica. Por fim, o quarto capítulo traz as considerações finais, abordando os principais resultados e contribuições alcançados, bem como os trabalhos futuros para a continuidade e melhoria do projeto.

2. Embasamento Teórico

Este capítulo apresenta os conceitos e tecnologias que serviram de base para o desenvolvimento das soluções apresentadas. A fundamentação abrange desde o detalhamento das ferramentas de backend e infraestrutura utilizadas para sustentar o ecossistema de automação do programa de capacitação.

2.1 O Framework Django e o Ecossistema Python

O framework *Django* é definido como uma estrutura de desenvolvimento web de alto nível, escrita em linguagem *Python*, que privilegia a rapidez no desenvolvimento e o design limpo e pragmático. Essa ferramenta permite que os desenvolvedores foquem na escrita do aplicativo sem a necessidade de reinventar componentes básicos, abstraindo tarefas repetitivas do desenvolvimento web e garantindo, nativamente, uma base sólida e protegida contra vulnerabilidades de segurança comuns (DJANGO SOFTWARE FOUNDATION, 2025).

O principal objetivo do *Django* é facilitar a criação de aplicações complexas e orientadas a dados, oferecendo uma filosofia de "baterias inclusas" que disponibiliza recursos essenciais, como sistema de autenticação, interface administrativa e mapeamento objeto-relacional (ORM). O propósito do *framework* é permitir que sistemas sejam construídos de forma escalável e segura, mantendo a manutenibilidade do código mesmo em projetos de grande porte, o que o torna a escolha ideal para infraestruturas de *backend* que exigem alta confiabilidade no processamento de informações (DJANGO SOFTWARE FOUNDATION, 2025).

Na prática, a aplicação do framework ocorre por meio da estruturação de modelos (*models*), visões (*views*) e roteamento de *URLs*, permitindo a separação clara entre a lógica de negócio e a persistência de dados (DJANGO SOFTWARE FOUNDATION, 2025). Essa abordagem modular, aliada ao uso de ferramentas complementares como o *Django REST Framework*, possibilita a exposição de endpoints seguros que podem ser consumidos por outros serviços (DJANGO REST FRAMEWORK, 2024).

2.2 Arquitetura e Desenvolvimento de APIs Restful

O desenvolvimento de *APIs* fundamentadas no estilo arquitetural *REST* permite a comunicação padronizada entre sistemas distintos através do protocolo de transferência de

hipertexto (FIELDING, 2000). No ecossistema *Django*, essa arquitetura é operacionalizada através da definição de modelos de dados que representam as entidades de negócio e do uso de *serializers*, responsáveis pela conversão desses objetos em formatos de intercâmbio amplamente aceitos, como o *JSON* (MELE, 2024).

O principal objetivo da arquitetura *RESTful* é promover a escalabilidade e a independência entre o cliente e o servidor, facilitando a integração de sistemas distribuídos e heterogêneos. Através de uma interface uniforme e do princípio de ausência de estado (*stateless*), o sistema garante que cada requisição contenha todos os parâmetros necessários para sua execução de forma independente (FIELDING, 2000). Na prática, a manipulação dos recursos ocorre por meio de métodos padronizados do protocolo *HTTP*, nos quais verbos como *GET*, *POST*, *PUT* e *DELETE* são utilizados para ler, criar, atualizar e excluir dados, respectivamente, estabelecendo uma estrutura lógica para o consumo da *API* (MELE, 2024).

Todo esse pacote de ferramentas para desenvolvimento da *API* vêm da biblioteca *django-rest-framework* na versão 3.15, essa biblioteca permite o desenvolvimento voltado apenas para o lado arquitetural *REST*, não sendo necessário uma conexão com o *frontend*. Essa organização lógica garante que os dados de autoavaliação dos discentes sejam processados e servidos de forma estruturada para alimentar as camadas de automação do projeto (DJANGO REST FRAMEWORK, 2024). O uso de *JSON Web Token* estabelece um padrão seguro para a transmissão de informações autenticadas através da geração de tokens de acesso temporários. Dessa forma, o sistema assegura que apenas requisições devidamente autorizadas tenham acesso aos endpoints da *API*, protegendo a integridade das informações e garantindo a confiabilidade das operações de inserção e consulta no banco de dados (BENDORAITIS, 2020).

2.3 Docker

A tecnologia de containerização através do *Docker* possibilita o empacotamento da aplicação e de todas as suas dependências em contêineres isolados, garantindo a replicabilidade do ambiente de execução independentemente da infraestrutura subjacente. A utilização de arquivos de configuração como o *Dockerfile* e o *docker-compose* permite gerenciar o ambiente de desenvolvimento e produção de forma padronizada, eliminando inconsistências de configuração e facilitando processos de *deploy* contínuo (MERKEL, 2014).

O principal objetivo da tecnologia de containerização é garantir a portabilidade e a consistência do software ao longo de todo o ciclo de vida de desenvolvimento, independentemente da infraestrutura subjacente. A utilização do *Docker* elimina as inconsistências entre os ambientes de desenvolvimento, homologação e produção, assegurando que a aplicação se comporte da mesma maneira em qualquer servidor (Pahl, 2015). Além disso, a ferramenta promove o isolamento entre processos, facilitando a escalabilidade do sistema e a implementação de arquiteturas modernas, onde diferentes componentes podem ser gerenciados de forma independente e segura.

Na prática, a configuração e a orquestração desses ambientes são realizadas por meio de arquivos de texto estruturados que automatizam a montagem das imagens e dos serviços. A utilização de arquivos como o *Dockerfile* permite definir as camadas da aplicação, enquanto o *docker-compose* possibilita gerenciar múltiplos contêineres de forma padronizada (MERKEL, 2014).

2.4 PostgreSQL

Para a persistência robusta e confiável das informações, a utilização de sistemas de gerenciamento de banco de dados relacionais é essencial. O *PostgreSQL* destaca-se por sua alta conformidade com padrões e robustez no tratamento de dados complexos, permitindo o armazenamento estruturado do histórico de avaliações dos discentes. Essa camada de dados garante a rastreabilidade necessária para que a *API* forneça as métricas comparativas fundamentais para analisar a evolução das competências ao longo dos períodos avaliativos (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

O *PostgreSQL* é classificado tecnicamente como um sistema de gerenciamento de banco de dados objeto-relacional (*SGBD*) de código aberto que utiliza e amplia a linguagem *SQL*. O objetivo deste sistema é fornecer um ambiente confiável para a manipulação de dados, garantindo propriedades fundamentais de transação, conhecidas como ACID (Atomicidade, Consistência, Isolamento e Durabilidade) (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

A implementação e o uso do *PostgreSQL* ocorrem por meio da criação de esquemas lógicos que definem tabelas, relacionamentos e restrições de integridade que organizam o fluxo da informação. O funcionamento do banco baseia-se na modelagem de entidades e atributos que são traduzidos em estruturas relacionais para garantir a eficiência no acesso aos dados (ELMASRI; NAVATHE, 2018).

2.5 Integração de Sistemas e Ferramenta n8n

A integração fluida entre plataformas heterogêneas é o que viabiliza a orquestração de fluxos de trabalho automatizados. O *n8n* atua como uma ferramenta de automação de código aberto que simplifica a conexão entre serviços e *APIs* através de uma interface visual orientada a nós (N8N, 2025).

O objetivo principal dessa ferramenta *n8n* é permitir a criação de fluxos de trabalho complexos de forma altamente flexível e auto-hospedada, garantindo total controle sobre a privacidade e o tráfego dos dados institucionais. A plataforma visa democratizar a automação de processos ao permitir que usuários conectem diversos aplicativos por meio de gatilhos (*triggers*) e ações, facilitando a movimentação e transformação de dados entre diferentes ecossistemas de software sem a necessidade de codificação extensa para cada integração individual (N8N, 2025).

Na prática, a automação é construída por meio de uma interface gráfica onde os "nós" representam funções específicas, como requisições *HTTP*, filtros lógicos ou integração com serviços de terceiros. O funcionamento baseia-se na configuração sequencial desses componentes, onde a saída de um nó serve como entrada para o próximo, permitindo o processamento e a manipulação de informações em tempo real (YAP, 2022).

3. Atividades Realizadas

Este capítulo detalha as atividades desenvolvidas durante o período de estágio, compreendendo o desenvolvimento técnico da API, a estruturação da infraestrutura de dados e as integrações sistêmicas realizadas para o aprimoramento da infraestrutura de backend voltada ao monitoramento de soft skills, visando a centralização da gestão de dados e a automação de processos fundamentais para o apoio à decisão pedagógica.

3.1 Gerenciamento do Projeto

O gerenciamento de projetos constitui uma prática fundamental para assegurar o êxito no desenvolvimento de software, organizando as etapas de planejamento, execução e controle através da aplicação de conhecimentos e técnicas consagradas (PMI, 2017). A utilização de metodologias ágeis, como o Scrum e o Kanban, favorece uma gestão visual e iterativa do fluxo de trabalho, o que auxilia as equipes na identificação de gargalos e na otimização da entrega de valor (SCRUM, 2025). Complementarmente, a automação de processos surge como uma estratégia crucial para converter tarefas manuais repetitivas em rotinas computacionais coordenadas, reduzindo erros humanos e liberando os profissionais para atividades de caráter estratégico e criativo (GROOVER, 2010).

Dessa forma, o gerenciamento foi realizado seguindo os princípios da metodologia ágil *Scrum*, com a organização de ciclos de desenvolvimento e reuniões periódicas para o alinhamento de metas e priorização de requisitos. O fluxo de trabalho foi conduzido de forma iterativa, permitindo ajustes rápidos na arquitetura da API e nos fluxos de automação conforme as necessidades pedagógicas do programa Capacitação 4.0 eram identificadas.

A equipe envolvida era composta por 11 integrantes com diferentes níveis de especialidade. A coordenação, orientação técnica e mentorias de soft skills e práticas hands on, foram realizadas por 4 professores pesquisadores. O suporte acadêmico e de pesquisa contou com 2 discentes de mestrado. As mentorias especializadas em soft skills e práticas hands on foram orientadas por 2 professores pesquisadores especialistas na área.. Por fim, o corpo de pesquisadores de graduação era composto por 4 discentes.

Os papéis assumidos por este pesquisador envolveram especificamente a atuação como desenvolvedor backend e integrador de sistemas. O foco principal das atividades concentrou-se na modelagem do banco de dados, construção da API *Django* e orquestração dos fluxos de trabalho no *n8n*. Dessa forma, a responsabilidade central foi garantir a

sustentação técnica necessária para transformar as regras de negócio em soluções automatizadas de certificação e recomendação.

3.2 Planejamento e Levantamento de Requisitos

O processo de desenvolvimento teve início com a análise das necessidades operacionais do programa de Capacitação 4.0, especificamente no que tange ao acompanhamento do desenvolvimento socioemocional dos discentes. Durante o primeiro ciclo do projeto, iniciado sob o edital de 2024, identificou-se que a gestão das avaliações de *soft skills* ocorria de forma descentralizada e manualmente, baseada em planilhas eletrônicas que dificultavam a análise histórica e a correlação de dados entre os diferentes períodos avaliativos. O requisito primordial estabelecido foi a criação de uma camada de persistência centralizada que pudesse atuar como uma fonte única de dados para todos os fluxos de automação subsequentes.

Para o levantamento dos requisitos funcionais, foram rigorosamente seguidas as diretrizes normativas estabelecidas no Manual da EMBRAPII (2024), documento que fundamenta a matriz complexa de competências do programa, composta por sete soft skills e quatro sub-soft skills. A *API* foi projetada para receber notas numéricas em uma escala de zero a quatro, aplicando automaticamente as regras de negócio para a conversão desses valores em níveis qualitativos: abaixo do básico, básico, adequado e avançado. A aderência estrita a este manual foi crucial para garantir a padronização dos relatórios gerados, eliminando a subjetividade e mitigando potenciais erros de cálculo manual que poderiam comprometer a avaliação por parte dos mentores.

Com a transição para o segundo ciclo do projeto, sob o edital de 2025, novos requisitos foram incorporados ao planejamento. Além do armazenamento e processamento das notas, o sistema passou a demandar funcionalidades voltadas para a automação da emissão de certificados e o fornecimento de informações estruturadas para motores de recomendação de práticas *hands-on*. Foi definido que a *API* deveria expor *endpoints* específicos que permitissem à ferramenta *n8n* recuperar não apenas a nota bruta, mas o perfil evolutivo completo do aluno, possibilitando a criação automática de gráficos radar comparativos e a verificação automática dos critérios de certificação, que exigem a evolução de nível em pelo menos dois terços das competências avaliadas.

No que tange aos requisitos não funcionais, estabeleceu-se a necessidade de uma arquitetura que garantisse a segurança e a integridade dos dados institucionais. Como a *API*

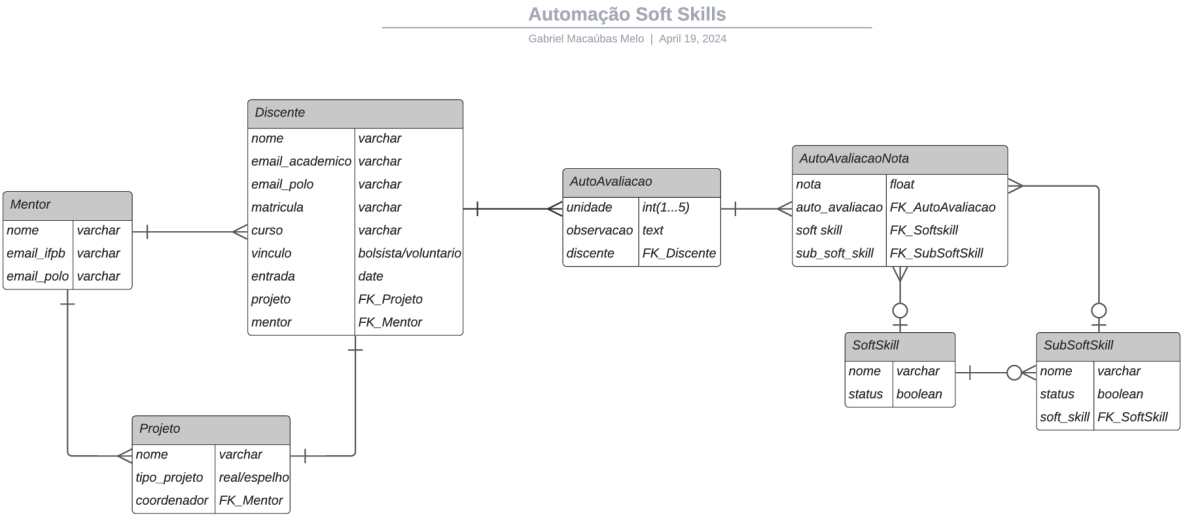
seria consumida por ferramentas externas através da internet, a implementação de um sistema de autenticação robusto tornou-se um requisito obrigatório. Além disso, a portabilidade do ambiente de desenvolvimento e a facilidade de *deploy* foram identificadas como pontos críticos, visto que a solução deveria ser replicável e de fácil manutenção por outros membros do time de pesquisa. Esses elementos de planejamento serviram de base para a escolha das tecnologias e para a estruturação de todo o cronograma de desenvolvimento técnico que será detalhado nas seções subsequentes.

3.3 Modelagem e Arquitetura do Banco de Dados

A estruturação da camada de persistência foi um dos pilares técnicos mais críticos do projeto, uma vez que a fidelidade dos relatórios e a rastreabilidade da evolução dos alunos dependiam diretamente de uma modelagem de dados íntegra e bem relacionada. Optou-se pela utilização do sistema de gerenciamento de banco de dados relacional *PostgreSQL*, aproveitando sua robustez no tratamento de chaves estrangeiras e a facilidade de integração com o *Object-Relational Mapping* do *Django*. A arquitetura foi desenhada para refletir a hierarquia organizacional do programa de capacitação, permitindo que cada registro de avaliação estivesse estritamente vinculado a um discente, a um projeto específico e a um mentor responsável.

Na Figura 1, apresenta-se o diagrama entidade-relacionamento que detalha a estrutura lógica do banco de dados utilizado no desenvolvimento da *API*. Nesse exemplo, visualiza-se a modelagem das entidades discente, mentor e projeto, bem como a definição técnica das chaves estrangeiras que conectam as autoavaliações às competências socioemocionais processadas. A Figura 1 representa visualmente a hierarquia dos dados e como as tabelas se relacionam para garantir a integridade referencial de todos os registros armazenados pelo sistema de maneira eficiente.

Figura 1: Diagrama Entidade-Relacionamento da estrutura do banco de dados



Fonte: Autoria própria, 2024

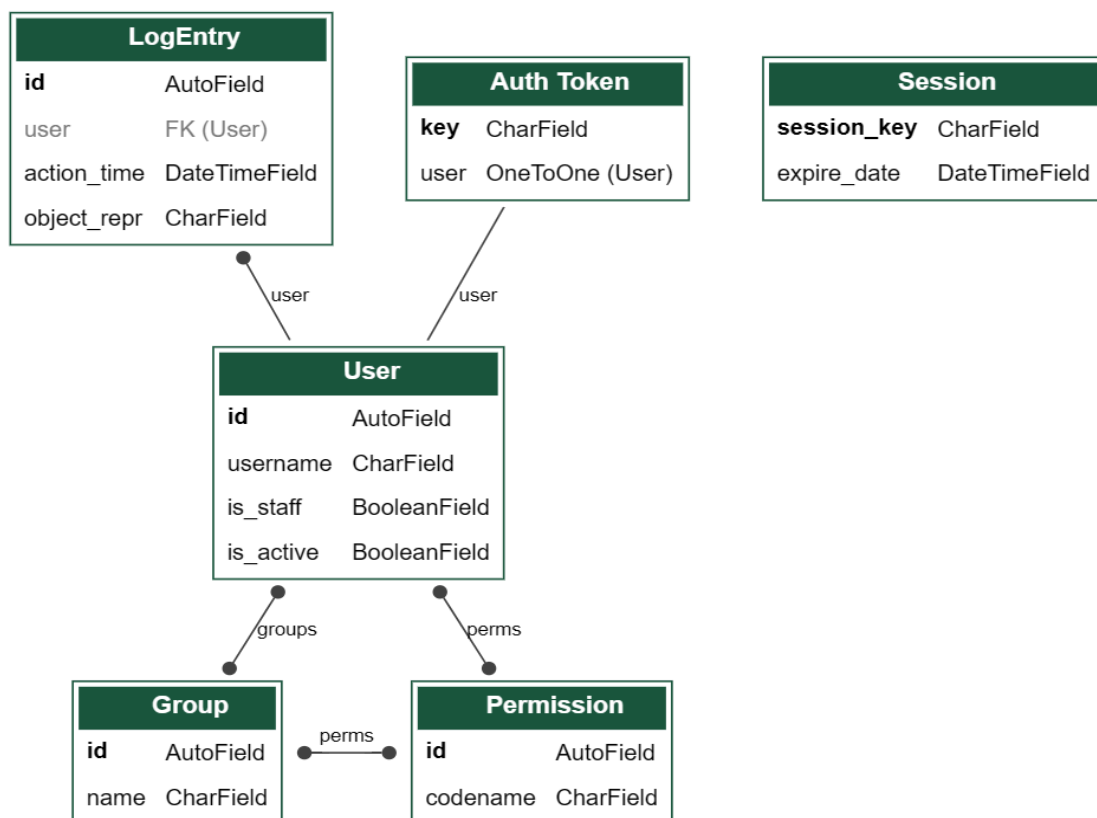
No centro da modelagem encontram-se as entidades discente e mentor, que armazenam informações de identificação e vínculo institucional. A tabela de discentes foi estruturada para conter dados como nome completo, correios eletrônicos acadêmicos e do polo, além do Cadastro de Pessoas Físicas, garantindo a unicidade de cada participante no sistema. Paralelamente, a entidade projeto permite classificar as atividades entre projetos reais ou espelhos, conforme as diretrizes da metodologia de aprendizagem baseada em problemas, vinculando cada iniciativa a um coordenador por meio de relacionamentos de integridade referencial. Essa organização possibilita que a *API* realize consultas complexas para filtrar o desempenho de alunos por turmas ou áreas de atuação específicas dentro do Polo de Inovação.

A lógica das competências socioemocionais foi implementada através de uma estrutura hierárquica composta pelas tabelas de *soft skills* e *subsoft skills*. Essa separação permite que uma competência macro, como o pensamento crítico, possa englobar diversas habilidades menores, mantendo a flexibilidade para futuras expansões na matriz de competências. Cada *subsoft skill* possui um vínculo direto com sua *soft skill* pai, o que facilita o processo de agregação de notas durante a geração dos cálculos qualitativos. Para suportar os diferentes momentos avaliativos previstos no manual da EMBRAPII, a

entidade de autoavaliação foi projetada para registrar o período específico, seja ele inicial, intermediário ou final, permitindo o armazenamento de observações textuais que complementam a análise quantitativa das notas.

Adicionalmente às tabelas do projeto, o ecossistema conta com uma estrutura robusta de tabelas nativas do *Django*, essenciais para a sustentação e governança da aplicação. Na Figura 2, apresenta-se o diagrama das entidades de suporte do *framework*, que gerenciam funcionalidades críticas como o controle de sessões, o registro de *logs* de auditoria e, primordialmente, o sistema de autenticação e autorização. Essa estrutura fornece a base necessária para que os protocolos de segurança *JWT* funcionem corretamente, vinculando cada operação de escrita na *API* a um usuário devidamente identificado e com as permissões adequadas.

Figura 2: Estrutura de tabelas nativas do Django para suporte e governança do sistema



Fonte: Autoria própria, 2025

3.4 Desenvolvimento do Backend e Lógica de Negócio

O desenvolvimento do backend foi realizado utilizando o *framework Django*, aproveitando sua estrutura de alto nível para garantir a agilidade na implementação e a segurança no tratamento das requisições. A construção do sistema iniciou-se pela definição dos modelos de dados, onde cada classe *Python* foi projetada para representar fielmente as entidades de negócio levantadas na fase de planejamento. A utilização do Object-Relational Mapping (ORM) do Django permitiu que as interações com o banco de dados *PostgreSQL* ocorressem de forma abstrata e segura. Para otimizar o desempenho do sistema e evitar o problema comum de múltiplas consultas redundantes, conhecido como N+1, foram aplicadas as técnicas de *select_related* e *prefetch_related* no carregamento de objetos relacionados, garantindo que informações de discentes, projetos e competências fossem recuperadas de maneira eficiente em um número reduzido de consultas ao banco de dados.

A inteligência da aplicação reside na camada de *serializers* e na lógica de negócio, responsáveis por transformar dados brutos em informações interpretáveis pelos fluxos de automação. Nos *serializers* implementados, foi desenvolvida a lógica necessária para organizar as notas das autoavaliações, utilizando campos de método para filtrar e separar as competências macro das *subsoft skills*. Esta camada não apenas valida a integridade dos dados recebidos, mas também garante que o formato de saída seja compatível com as necessidades de processamento gráfico. Através dessa estruturação, a API assegura que cada registro de nota esteja corretamente vinculado à sua respectiva categoria, permitindo uma análise granular do desempenho do aluno.

Para gerenciar o fluxo de dados e as operações de escrita, foram desenvolvidas visões especializadas que concentram a lógica de criação e resposta do sistema. As classes de *ViewSet* foram configuradas para exigir autenticação e para processar requisições do tipo *POST*, onde o sistema recebe as informações de autoavaliação e persiste os dados. Visando a eficiência na gravação de registros volumosos, como as múltiplas notas de competências enviadas simultaneamente, utilizou-se a técnica de *bulk_create*. Este método permitiu que todas as instâncias de notas fossem inseridas no banco de dados através de uma única transação, reduzindo drasticamente o tempo de resposta da *API* e o overhead de comunicação com o servidor. A resposta personalizada reconstrói o nome do aluno, a unidade avaliativa e um mapeamento dinâmico das notas por competência, fornecendo um

retorno estruturado que serve de gatilho para as etapas subsequentes do fluxo de automação no *n8n*.

Na Figura 3, apresenta-se uma composição técnica que ilustra a integração entre o modelo de dados e a visão da aplicação, constituindo o núcleo funcional do sistema. O leitor deve observar, inicialmente, (1) os decoradores de autenticação e permissão no topo da *AutoavaliacaoViewSet*, que garantem que apenas requisições autenticadas via *Token* ou Sessão acessem ao recurso. No método *create*, destaca-se (2) a lógica de validação de dados (*is_valid()*), que assegura a integridade antes da persistência.

Adicionalmente, visualiza-se (3) o mapeamento dinâmico de notas no método *get_notas*, responsável por transformar os objetos do banco de dados em pares de chave-valor (competência: nota) para o *JSON* de resposta. Na camada de modelo, a classe *AutoavaliacaoNota* evidencia (4) a estrutura de chaves estrangeiras e a integridade referencial (*models.PROTECT*), além dos campos de auditoria que registam automaticamente o autor e a data de cada atualização. A Figura 3 representa, portanto, o fluxo completo: desde a barreira de segurança e validação até à entrega de uma resposta estruturada para o consumo imediato pelas ferramentas de integração.

Figura 3: Composição técnica de Model e ViewSet da API Django

```

@authentication_classes([SessionAuthentication, TokenAuthentication])
@permission_classes([permissions.IsAuthenticated])
class AutoavaliacaoViewSet(ModelViewSet):
    http_method_names = ['post']

    def get_serializer_class(self):
        if self.request.method == 'POST':
            return CreateAutoavaliacaoSerializer
        return ListAutoavaliacaoSerializer

    def get_queryset(self):
        _queryset = Discente.objects.get(
            id=self.request.GET.get('discente', None)
        ) if 'discente' in self.request.GET else Discente.objects.none()

        return _queryset

    def get_notas(self, autoavaliacao, _response):
        for nota_objeto in autoavaliacao.notas.filter(soft_skill__isnull=False):
            response[nota_objeto.soft_skill.nome] = nota_objeto.nota
        return _response

    def create(self, request, *args, **kwargs):
        _data = request.data
        _serializer = self.get_serializer_class()
        _serializer_data = _serializer(data=_data)

        if _serializer_data.is_valid():
            _autoavaliacao = _serializer_data.save(usuario_logado=request.user)
            _autoavaliacao.cadastrado_por = request.user
            _autoavaliacao.atualizado_por = request.user
            _autoavaliacao.save()
            _serializer_response = _serializer(instance=_autoavaliacao)
            _response = {
                "aluno": _autoavaliacao.discente.nome,
                "autoavaliacao": _autoavaliacao.unidade
            }
            _response = self.get_notas(_autoavaliacao, _response)

        return Response(_response, status=HTTP_201_CREATED)

class AutoavaliacaoNota(models.Model):
    # campos
    nota = models.DecimalField(max_digits=8, decimal_places=2, null=True, default=0)
    autoavaliacao = models.ForeignKey(
        to='capitacao.Autoavaliacao', on_delete=models.PROTECT, related_name='notas', null=True
    )
    soft_skill = models.ForeignKey(
        to='capitacao.SoftSkill', on_delete=models.PROTECT, related_name='notas', null=True, blank=True
    )
    sub_soft_skill = models.ForeignKey(
        to='capitacao.SubSoftSkill', on_delete=models.PROTECT, related_name='notas', null=True, blank=True
    )
    cadastrado_em = models.DateTimeField(auto_now_add=True)
    atualizado_em = models.DateTimeField(auto_now=True)
    cadastrado_por = models.ForeignKey(
        to='auth.User', on_delete=models.PROTECT, related_name='notas_cadastradas', null=True
    )
    atualizado_por = models.ForeignKey(
        to='auth.User', on_delete=models.PROTECT, related_name='notas_atualizadas', null=True
    )

    class Meta:
        verbose_name = 'Nota da Autoavalição'
        verbose_name_plural = 'Notas das Autoavaliações'

    def __str__(self):
        return (f'{self.autoavaliacao.discente}: {self.nota} - '
            + (str(self.sub_soft_skill) if self.sub_soft_skill else str(self.soft_skill)))

```

Fonte: Autoria própria, 2025

3.5 Segurança, Autenticação e Documentação da API

A garantia da segurança e da integridade dos dados foi tratada como um requisito transversal e prioritário em todas as etapas de desenvolvimento da *API*. Considerando que o sistema lida com informações sensíveis de desempenho discente e que os endpoints são expostos para consumo via internet por ferramentas externas como o *n8n*, a implementação de mecanismos robustos de controle de acesso tornou-se indispensável. Para atender a essa necessidade, optou-se pela utilização do padrão *JSON Web Token* através da integração com o *Django Rest Framework*, estabelecendo um modelo de autenticação baseada em tokens que permite a comunicação segura e sem estado entre as aplicações.

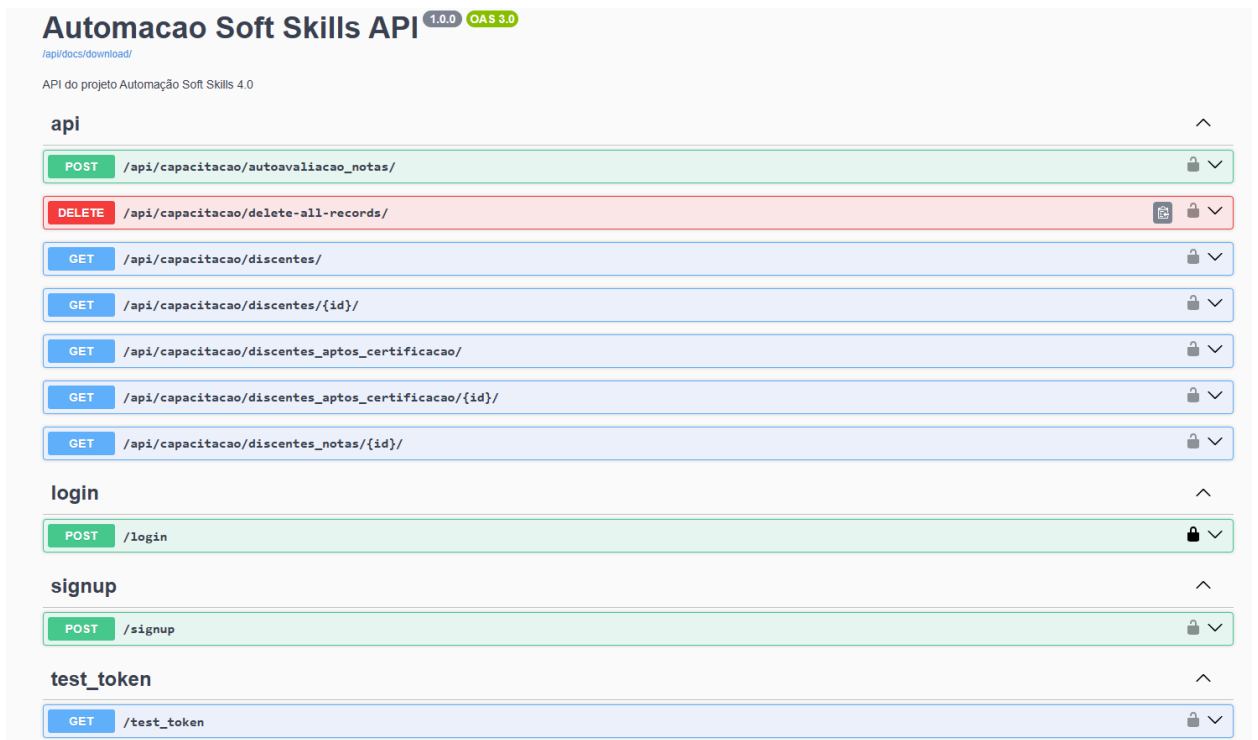
A proteção das rotas foi operacionalizada através do uso rigoroso de classes de autenticação e permissão diretamente nas *ViewSet*s do sistema. Ao configurar decoradores de classes de autenticação, como o *TokenAuthentication* e o *SessionAuthentication*, a *API* passou a exigir que cada requisição contenha um cabeçalho de autorização válido para ser processada. Complementarmente, a aplicação de permissões do tipo *IsAuthenticated* garantiu que apenas usuários devidamente cadastrados e autenticados no sistema pudessem realizar operações de escrita ou consulta no banco de dados. Essa camada de proteção é fundamental para prevenir acessos não autorizados e assegurar que a manipulação das

notas de soft skills ocorra exclusivamente dentro do escopo de operação do time de pesquisa.

Além da segurança em nível de aplicação, a gestão de informações sensíveis foi reforçada através de boas práticas de infraestrutura, como o uso de variáveis de ambiente para o armazenamento de chaves secretas e credenciais de banco de dados. No código-fonte, evitou-se a exposição direta de dados críticos, utilizando arquivos de configuração que segregam as definições de ambiente de desenvolvimento das definições de produção. Essa abordagem, aliada ao uso do *framework Django*, que já oferece proteções nativas contra vulnerabilidades comuns como *Cross-Site Request Forgery* e *SQL Injection*, resultou em um ambiente tecnológico resiliente e confiável para a sustentação do projeto.

Para facilitar a integração sistêmica e garantir a transparência do desenvolvimento, a *API* foi integralmente documentada utilizando o padrão *OpenAPI*. Através da geração automática de esquemas, foi possível disponibilizar uma interface interativa que detalha todos os endpoints, parâmetros de entrada e formatos de resposta esperados. Durante a fase de construção, a ferramenta *Insomnia* foi utilizada de forma complementar para realizar testes nos endpoints, validando o comportamento das rotas e a estrutura dos dados antes da integração final com os fluxos automatizados. Na Figura 4, apresenta-se a interface de documentação da *API*, onde se observa a listagem organizada das rotas disponíveis para discentes, projetos e avaliações. A Figura 4 representa visualmente o contrato de comunicação do sistema, permitindo que outros desenvolvedores ou ferramentas de automação compreendam a estrutura de dados e testem as funcionalidades de maneira eficiente, o que acelerou significativamente o processo de integração com os fluxos de trabalho no *n8n*.

Figura 4: Interface de documentação técnica e endpoints da API Django



Fonte: Autoria própria, 2025

3.6 Integração com n8n e Automação de Fluxos

A integração entre a *API* desenvolvida e a plataforma de automação n8n constituiu o estágio de operacionalização de toda a inteligência de dados do projeto. Enquanto o backend em *Django* foi responsável por garantir a integridade e a lógica de processamento das informações, o *n8n* atuou como o orquestrador dos fluxos de trabalho, conectando a camada de persistência a ferramentas de produtividade como o *Google Sheets* e o *Google Drive*. Essa arquitetura permitiu que processos manuais exaustivos, como a leitura de formulários de autoavaliação e a transposição de dados para relatórios individuais, fossem substituídos por fluxos digitais contínuos e escaláveis.

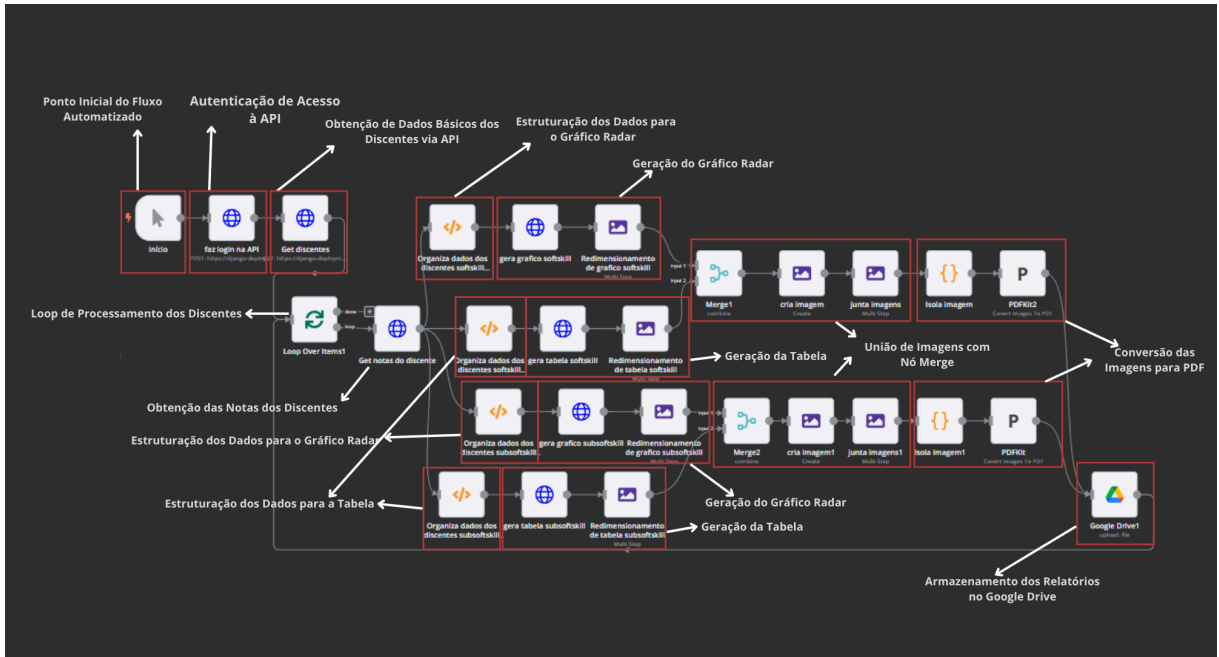
No fluxo de trabalho estabelecido para o primeiro ciclo do projeto, a integração ocorria de forma que o n8n monitorava a entrada de novas respostas em planilhas de desempenho. Ao identificar um novo registro, a automação realizava uma chamada via método *POST* para o endpoint correspondente na *API Django*, enviando os dados brutos e as notas atribuídas pelos discentes. O sistema então processava a requisição, validava a autenticidade por meio dos cabeçalhos de autorização e armazenava os registros no banco de dados. Após o processamento, o *n8n* recuperava as informações já calculadas e formatadas para alimentar modelos de documentos, resultando na criação automática de

relatórios em PDF com gráficos radar que ilustram o perfil socioemocional de cada estudante.

Durante o segundo ciclo, sob o edital de 2025, a complexidade da integração foi ampliada para contemplar a automação da emissão de certificados e a geração de recomendações de práticas PBL. Nesse cenário, a *API* passou a fornecer respostas estruturadas que indicavam se o discente havia cumprido os critérios de evolução mínima exigidos pela EMBRAPIL. O fluxo no *n8n* foi configurado para interpretar esses indicadores lógicos e, em caso de conformidade, acionar os nós de geração de documentos e envio de correios eletrônicos automáticos. Essa sinergia entre o código customizado e a ferramenta de automação garantiu que as regras de certificação fossem aplicadas de maneira uniforme, aumentando a transparência e a agilidade da governança do programa.

Na Figura 5, apresenta-se o fluxo de automação estruturado no *n8n*, demonstrando a sequência de nós responsáveis por orquestrar a comunicação com a *API*. Nesse exemplo, visualiza-se o nó de requisição *HTTP* configurado para interagir com os endpoints do *Django*, seguido pelos nós de processamento de dados e armazenamento no *Google Drive*. A Figura 5 representa visualmente como os diferentes serviços se conectam para executar as tarefas de maneira eficiente, ilustrando o papel do *n8n* como o cliente principal que consome os serviços da *API* para transformar dados brutos em produtos finais, como planilhas atualizadas e certificados digitais devidamente organizados.

Figura 5: Fluxo de automação no n8n integrado aos endpoints da API Django



Fonte: Autoria própria, 2025

3.7 Infraestrutura, Containerização e Deploy

A etapa final do desenvolvimento técnico concentrou-se na preparação da infraestrutura necessária para suportar a aplicação em ambientes de homologação e produção, garantindo a disponibilidade dos serviços para os fluxos de automação. Para assegurar que a *API* funcionasse de maneira idêntica em diferentes servidores e computadores da equipe de pesquisa, eliminando o problema de inconsistência de ambientes, adotou-se a estratégia de containerização. Esse processo permitiu o isolamento completo das dependências de software, garantindo que o comportamento da aplicação em nuvem fosse um reflexo exato do que foi validado durante a fase de desenvolvimento local.

A implementação do *Docker* foi estruturada através da criação de um arquivo de definição de imagem, o *Dockerfile*, onde se configurou um ambiente de execução baseado em *Python 3.9*. Nesse arquivo, foram estabelecidas as instruções para a definição do diretório de trabalho, a cópia do código-fonte e a instalação automatizada das bibliotecas necessárias através do gerenciador de pacotes. Complementarmente, a utilização do arquivo *docker-compose.yml* permitiu orquestrar a execução da *API* de forma coordenada, automatizando tarefas rotineiras como a aplicação de migrações no banco de dados e a

inicialização do servidor, o que facilitou significativamente a entrada de novos membros no projeto de pesquisa.

Na Figura 6, apresenta-se a configuração de containerização da aplicação, evidenciando os três pilares que garantem a replicabilidade do sistema. O leitor deve observar (1) o arquivo *Dockerfile*, que contém as instruções sequenciais para a construção da imagem, incluindo a definição da imagem base *Python 3.12* e a preparação do ambiente interno. Complementarmente, destaca-se (2) o *docker-compose.yml*, responsável pela orquestração dos serviços, onde são geridas as interdependências entre o servidor *Django* e o banco de dados *PostgreSQL*, além da gestão de volumes e mapeamento de portas.

Por fim, visualiza-se (3) o arquivo *requirements.txt*, que centraliza a gestão de dependências do ecossistema Python, garantindo que todas as bibliotecas sejam instaladas nas versões exatas em qualquer ambiente. A Figura 6 representa visualmente a abstração da infraestrutura, demonstrando como a integração dessas ferramentas assegura que o backend opere de maneira consistente e eficiente entre os ambientes de desenvolvimento local e produção.

Figura 6: Configurações de Docker

The image displays three distinct Docker configuration snippets, each enclosed in a red rectangular border and marked with a red number.
Snippet 1 (left) defines a multi-service Docker Compose configuration. It includes a 'db' service using the 'postgres:15' image, with environment variables for database name, user, and password, and a 'web4' service using the 'automacao_deploy' image, which runs Django management commands and listens on port 8000.
Snippet 2 (top right) shows a Dockerfile for a Python 3.12 environment. It sets the Python unbuffered environment, the working directory to /app, updates the system and installs development tools (libpq-dev, gcc), cleans up, copies requirements, and installs specific versions of pip and dependencies.
Snippet 3 (bottom right) lists a comprehensive set of Python dependencies and their versions, including Django, Django REST framework, and various utility libraries like psycopg2, pyyaml, and typing_extensions.

```
version: '3'
services:
  db:
    image: postgres:15
    container_name: automacao_deploy_db
    environment:
      - POSTGRES_DB=${DB_NAME}
      - POSTGRES_USER=${DB_USER}
      - POSTGRES_PASSWORD=${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
  web4:
    container_name: 'automacao_deploy'
    build: .
    command: >
      sh -c "python manage.py migrate &&
        python manage.py runserver 0.0.0.0:8000"
    ports:
      - "8000:8000"
    volumes:
      - ./app
    depends_on:
      - db
    environment:
      - DB_NAME=${DB_NAME}
      - DB_USER=${DB_USER}
      - DB_PASSWORD=${DB_PASSWORD}
      - DB_HOST=${DB_HOST}
      - DB_PORT=${DB_PORT}
    volumes:
      postgres_data:
```

```
FROM python:3.12

ENV PYTHONUNBUFFERED 1

WORKDIR /app

RUN apt-get update && apt-get install -y \
  libpq-dev \
  gcc \
  && rm -rf /var/lib/apt/lists/*

COPY requirements.txt /app/requirements.txt
RUN pip3 install pip==25.1.1 && pip3 install -r requirements.txt

COPY . /app
```

```
asgiref==3.8.1
attrs==23.2.0
dj-database-url==2.2.0
Django==4.2.11
django-extensions==4.1
django-filter==24.2
django-rest-framework==3.15.1
drf-spectacular==0.27.2
drf-yasg==1.21.7
importlib-metadata==7.1.0
inflection==0.5.1
jsonschema==4.21.1
jsonschema-specifications==2023.12.1
Markdown==3.6
packaging==24.0
psycopg2-binary==2.9.9
python-decouple==3.8
python-dotenv==1.0.1
pytz==2024.1
PyYAML==6.0.1
referencing==0.35.0
rpds-py==0.18.0
sqlparse==0.5.0
typing_extensions==4.11.0
tzdata==2024.1
uritemplate==4.1.1
zipp==3.18.1
```

Autoria própria, 2025

O deploy da aplicação foi realizado na plataforma *Vercel*, utilizando um fluxo de integração contínua conectado ao repositório de código *automacao-deploy*. Para adaptar o *framework Django* às especificidades de execução *serverless* da plataforma, foi necessária a configuração do arquivo *vercel.json*, que orienta o servidor sobre como rotear as requisições para a interface de *gateway* da aplicação. Essa configuração permitiu que a *API* operasse de forma escalável e com alta disponibilidade, servindo como o ponto de acesso público estável para que as automações do *n8n* pudessem realizar chamadas de rede e processar as notas de *soft skills* sem interrupções.

Para assegurar a proteção da infraestrutura em ambiente de produção, toda a gestão de dados sensíveis foi realizada através de variáveis de ambiente configuradas de forma segura. Isso garantiu que informações críticas, como a chave secreta do *framework* e as credenciais de acesso ao banco de dados *PostgreSQL*, não ficassem expostas no código-fonte público. A integração entre a portabilidade oferecida pelo *Docker* e a agilidade do deploy na *Vercel* resultou em um ecossistema técnico robusto, capaz de

sustentar as operações de monitoramento e certificação do programa Capacitação 4.0 com segurança e alto desempenho profissional.

Na Figura 7, apresenta-se a configuração técnica necessária para o *deploy* da aplicação na plataforma *Vercel*, evidenciando os dois componentes fundamentais que sustentam o ambiente serverless. À esquerda da imagem, observa-se o arquivo *vercel.json*, responsável por detalhar as diretrizes de roteamento e o runtime de execução que permite a comunicação entre a web e a lógica do *framework Django*. À direita, destaca-se o *script build.sh*, que orquestra os comandos de construção automatizada, garantindo a coleta de arquivos estáticos e a instalação de dependências essenciais no servidor de produção. Esta composição ilustra de maneira integrada o processo de entrega contínua, demonstrando como a infraestrutura de nuvem é preparada para disponibilizar a *API* de forma segura e escalável para o consumo pelos fluxos de automação do *n8n*.

Figura 7: Parâmetros de deploy na plataforma Vercel



```
{
  "version": 2,
  "builds": [
    {
      "src": "core/wsgi.py",
      "use": "@vercel/python",
      "config": {
        "maxLambdaSize": "15mb",
        "runtime": "python3.9"
      }
    },
    {
      "src": "build.sh",
      "use": "@vercel/static-build",
      "config": {
        "distDir": "staticfiles_build"
      }
    }
  ],
  "routes": [
    {
      "src": "/static/(.*)",
      "dest": "/static/$1"
    },
    {
      "src": "/(.*)",
      "dest": "core/wsgi.py"
    }
  ]
}
```

```
#!/bin/bash

echo "BUILD START"
python3.9 -m ensurepip
python3.9 -m pip install -r requirements.txt
echo "BUILD END"

python3.9 manage.py migrate

echo "Collect Static..."
python3.9 manage.py collectstatic
```

Autoria própria, 2025

4. Considerações Finais

A realização deste estágio supervisionado, desenvolvido ao longo de dois ciclos consecutivos de pesquisa e inovação tecnológica, proporcionou uma experiência integral na aplicação prática dos conceitos fundamentais do curso de Tecnologia em Sistemas para Internet. A participação ativa nos editais de 2024 e 2025 permitiu não apenas a observação, mas a construção efetiva de uma solução robusta para os desafios de gestão de *soft skills* no programa Capacitação 4.0 da EMBRAPPII e do IFPB. Diferente de uma atuação isolada em apenas uma etapa do processo, o acompanhamento do ciclo completo de vida do software, desde a concepção da camada de persistência até a integração com ferramentas de automação de fluxos, consolidou uma visão sistêmica sobre como a tecnologia pode ser empregada para otimizar processos educacionais e profissionais complexos.

O desenvolvimento da *API Restful* utilizando o *framework Django* representou o pilar central das atividades realizadas, consolidando-se como o motor de uma transformação significativa na eficiência operacional do programa. Ao estruturar um backend capaz de manter a integridade de registros históricos e automatizar a tradução das métricas quantitativas do Manual de Operação da EMBRAPPII (2024) em avaliações qualitativas, foi possível alcançar ganhos de produtividade. O impacto dessa solução é evidenciado pela redução no tempo de processamento: enquanto o método manual anterior consumia aproximadamente uma hora por aluno para que o mentor realizasse todos os cálculos e a geração de planilhas, o sistema automatizado reduziu esse intervalo para apenas 2 a 3 segundos no registro das notas e cerca de 12 segundos para a geração de cada gráfico de desempenho. Esse resultado demonstra como o alinhamento entre o conhecimento técnico em Python e as diretrizes de governança do programa permitiu converter fluxos de trabalho extensos em rotinas automatizadas.

A maturidade técnica alcançada durante o estágio também se manifestou na gestão da infraestrutura e segurança da aplicação. A adoção da tecnologia *Docker* para a containerização do projeto garantiu a portabilidade e a padronização do ambiente de desenvolvimento, uma prática essencial para o mercado de trabalho atual. Paralelamente, a estruturação do banco de dados relacional *PostgreSQL* exigiu um rigoroso planejamento para assegurar a rastreabilidade das informações entre os períodos avaliativos. A proteção desses dados através da implementação de autenticação via *JSON Web Token (JWT)* reforçou a compreensão de que a segurança da informação deve ser uma prioridade nativa

no desenvolvimento de sistemas voltados à internet, especialmente em ambientes que lidam com dados institucionais e de desempenho discente.

Outro ponto de destaque nesta trajetória foi a integração sistêmica com a ferramenta de automação *n8n*. Enquanto no primeiro ciclo o foco residiu na estruturação da *API* para suporte à geração de gráficos e cadastros, o segundo ciclo demandou uma evolução do sistema para fornecer dados ainda mais granulares, permitindo a automação completa da emissão de certificados e o suporte a sistemas de recomendação. Essa experiência de integrar uma *API* customizada com plataformas low-code evidenciou o potencial de interoperabilidade entre sistemas, demonstrando como ferramentas heterogêneas podem trabalhar de forma coordenada para reduzir significativamente o esforço manual e a margem de erro em processos administrativos e acadêmicos.

O trabalho em equipe, guiado pelas metodologias ágeis *Scrum* e *Kanban*, foi fundamental para o cumprimento das metas estabelecidas nos planos de trabalho. A interação constante com mentores, coordenadores e outros estagiários permitiu o desenvolvimento de competências de comunicação e colaboração que são tão vitais quanto o conhecimento técnico. A necessidade de lidar com prazos rigorosos, entregas iterativas e a resolução de problemas imprevistos, como a configuração de variáveis de ambiente e o *deploy* na plataforma *Vercel*, preparou o terreno para uma transição segura da vida acadêmica para a atuação profissional plena como desenvolvedor *backend* e analista de sistemas.

Por fim, conclui-se que os objetivos propostos para o estágio e para o projeto de pesquisa foram atingidos. A *API* desenvolvida não é apenas um artefato de software, mas uma ferramenta ativa que agregou valor ao processo de gestão e monitoramento da avaliação de *Soft Skill* para o Programa de Capacitação 4.0 da EMBRAPPI no Polo de Inovação do IFPB. A vivência prática com o ecossistema *Python*, *Docker* e a integração com fluxos de automação proporcionou uma base técnica sólida e diferenciada. Esta jornada reafirmou a importância de uma formação acadêmica que valoriza o aprender fazendo, deixando como legado uma infraestrutura tecnológica funcional e um conhecimento profissional que certamente servirá de alicerce para os próximos desafios na área de tecnologia da informação.

Como perspectivas para trabalhos futuros, propõe-se a expansão da *API* para novos critérios avaliativos e o aprimoramento da integração com o *n8n* para maior escalabilidade.

Planeja-se o desenvolvimento de interfaces de usuário (*frontend*) dedicadas para substituir processos manuais em planilhas, além da implementação de módulos de inteligência artificial voltados à análise preditiva e ao refinamento das recomendações pedagógicas. Tais evoluções consolidarão o sistema como uma plataforma inteligente e modular de gestão de competências no Polo de Inovação do IFPB.

Referências

- BENDORAITIS, Aidas. *Django 3 Web Development Cookbook: solutions to develop, deploy, and manage your Python web applications*. 4. ed. Birmingham: Packt Publishing, 2020.
- DJANGO REST FRAMEWORK. *Django REST framework documentation*. Disponível em: <https://www.django-rest-framework.org/>. Acesso em: 01 fev. 2026.
- DJANGO SOFTWARE FOUNDATION. *Django Documentation*. Disponível em: <https://docs.djangoproject.com/en/4.2/>. Acesso em: 03 fev. 2026.
- ELMASRI, Ramez; NAVATHE, Shamkant B. *Sistemas de Banco de Dados*. 7. ed. Rio de Janeiro: Pearson, 2018.
- EMBRAPII. Manual do Programa Capacitação 4.0. v 1.0. Brasília: Empresa Brasileira de Pesquisa e Inovação Industrial, 2021.
- FIELDING, Roy Thomas. *Architectural Styles and the Design of Network-based Software Architectures*. 2000. Tese (Doutorado em Informação e Ciência da Computação) – University of California, Irvine, 2000.
- GROOVER, Mikell P. *Automação industrial e sistemas de manufatura*. 3. ed. São Paulo: Pearson Prentice Hall, 2010.
- MERKEL, Dirk. Docker: *lightweight Linux containers for consistent development and deployment*. Linux Journal, n. 239, p. 2, 2014.
- N8N. *N8N Documentation*. Disponível em: <https://docs.N8N.io>. Acesso em: 30 junho 2025.
- PAHL, Claus. *Containerization and the PaaS Cloud*. IEEE Cloud Computing, v. 2, n. 3, p. 24-31, 2015.
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. *Sistema de Banco de Dados*. 7. ed. Rio de Janeiro: Elsevier, 2020.
- SCRUM. *Scrum Guides – The Official Scrum Guide*. Disponível em: <https://scrumguides.org/>. Acesso em: 12 set. 2025.